

TC 자동화 시스템 개요

프로젝트별 TC(테스트 케이스) 설계부터 Playwright 자동 실행, 결함 관리까지 이어지는 QA 자동화 시스템입니다. 터미널/IDE에서 Claude Code를 직접 쓰거나(경로 A), 로컬 웹 대시보드에서 채팅·버튼으로 조작할 수 있습니다(경로 B). 2026-08-24 Slack 봇("큐돌이") 운영을 종료하고 이 두 경로로 완전히 전환했습니다. 아래는 이 시스템을 처음 보는 동료에게 설명하기 위한 구조·프로세스·사용법 정리입니다.

인터페이스: 터미널/IDE 직접 세션 + 대시보드 (:4000)

저장소: **tc-automation** (Git, 단일 저장소) 대시보드 구동: **Windows** 예약 작업 (**S4U**) 상시

최종 갱신: **2026-09-29**

목차

01 실행 경로 & 토큰 사용 영역

02 TC 생성·실행·결함관리 프로세스

03 대시보드 기능 지도

04 사용자 워크플로우

01 실행 경로 & 토큰 사용 영역

Claude를 실제로 호출(=비용 발생)하는 기능과, 코드만으로 결정론적으로 처리되어 호출하지 않는(zero-token) 기능을 나눴습니다. TC 설계처럼 실제 추론이 필요한 작업만 Claude를 쓰고, 이미 확정된 데이터를 반복 처리하는 작업은 대시보드가 코드로 직접 처리합니다.

기능	경로	토큰
TC 신규 생성·수정 (Phase 0~4, 정책서/URL 분석)	A: 터미널·IDE 세션 / B: 대시보드 채팅 패널	• 사용

결함관리 중 자유 형식 질문 ("이 결함 원인 이 뭐야?")	A 또는 B 채팅	● 사용
TC 업로드(엑셀/JSON) 완료 후 TC 뷰어 (HTML) 재생성	B → 완료 즉시 채팅에 자동 요청(백그라운드)	● 사용 (자동 트리거)
TC 엑셀·JSON 업로드 반영 (수정/신규 판 정, legacy 아카이브, 버전관리)	B: "+ TC 업로드"	● 미 사용
조건별 테스트 실행 (모듈/시스템/우선순 위/수행상태 필터, 또는 TC 뷰어 체크박스)	B: 테스트 실행 큐	● 미 사용
테스트 결과 → TC json / defects.json / results 스냅샷 반영	B: 코드가 직접 처리	● 미 사용
테스트 실행 화면 녹화 & 재생(영상·스크 린샷 팝업)	B: 테스트 실행 큐 "단계 재생" 모드	● 미 사용
결함 상태·담당자·이슈링크 변경	B: 결함 관리 표에서 직 접 편집	● 미 사용
git add + commit (테스트 실행/업로드 완료 시)	B: 자동	● 미 사용
git push	B: "🔗 Git" 버튼 — 사 용자가 직접 클릭해야만 실행	● 미 사용

02 TC 생성 → 실행 → 결함관리 프로세스

프로젝트 온보딩부터 결함 등록까지 이어지는 Phase 0~8 워크플로우(AGENTS.md 13항)입니다. 굵게 표시된 지점은 사람의 "승인"이 있어야만 다음 단계로 넘어갑니다 — 과거 Slack 자동연속 예외는 2026-08-24부로 전면 폐지되어, 어떤 경로로 요청하든 Phase당 1회씩 승인을 받습니다.

0

프로젝트 온보딩 & Phase 0 — 사전 질의

_template/을 project/{프로젝트명}/으로 복사 (Policy·SB·Requirements·TC). 단위/통합, 코드 기반/정책기반, URL 여부를 질의해 project.json에 저장 — 이후 요청부터는 재질의하지 않음.

1~3

Phase 1~3 — 근거 확보 & 시나리오 검토표

정책서 분석 또는 URL 관찰(Playwright) → 테스트 범위 산정 → 도출 가능한 전체 시나리오를 있는 그대로 제시

|| 승인 대기

4

Phase 4 — TC 상세 작성 & 산출물 생성

TC HTML/JSON 캐노니컬 파일 생성·갱신(legacy 아카이브 + meta.version/changeHistory).
URL 기반이면 Playwright 자동화 코드도 함께 생성(실행은 안 함)

|| 승인 대기

5

Phase 5 — 테스트 실행 (별도 요청 필요)

A: 터미널/IDE 세션이 직접 `npx playwright test` 실행 / B: 대시보드 테스트 실행 큐로 필터 조합 실행(zero-token, 실시간 로그·영상 녹화 옵션)

6~7

Phase 6~7 — 오류 공유 & 결함 관리

콘솔 에러·4xx/5xx는 자동으로 테스트를 실패시키고 스크린샷·콘솔로그를 확보. `defects.json`에 `tclid` 기준 신규/재발생/기존 실패 유지로 자동 판정 후 등록·갱신

8

Phase 8 — 결과 도출 & git 반영

Pass/Fail 요약 리포트 직후 자동 `git add project/{프로젝트명}` → `commit.push`는 항상 별도로 확인받은 뒤에만 실행(대시보드는 "🔗 Git" 버튼, 세션은 매번 재확인)

03 대시보드 기능 지도

로컬 웹 대시보드(`dashboard/` , Express + `ws` , 포트 4000)에서 프로젝트를 선택하면 아래 기능들을 쓸 수 있습니다. 표시 없는 기능은 전부 zero-token입니다.

** KPI · 결함 · 실행이력**

프로젝트별 결함 현황, 모듈별 수행율/Pass율, 우선순위 분포, 날짜별 실행 이력 스냅샷을 한 화면에서 확인

** 사이트분석 · TC 생성 (채팅)**

"큐돌이"에게 자연어로 TC 생성·수정·결함 문의 —
claude CLI를 헤드리스로 spawn(토큰 사용).
Phase당 승인 절차 동일 적용

▶ 테스트 실행 큐

모듈·시스템·우선순위·수행상태를 조합 선택(0건인 조합은 자동 비활성화) → Playwright 순차 실행 → 결과가 TC/defects.json/results에 자동 반영 + git 자동 커밋. "단계 재생" 모드로 실행 영상도 확인 가능

+ TC 업로드

엑셀(.xlsx) 또는 JSON 파일을 업로드하면 TC ID 기준으로 자동 수정/신규 판정. 같은 ID인데 분류·항목이 다르면 별개 케이스로 분리 생성(오답 방지 포함)

🔴 결함 관리

상태·담당자·이슈링크를 표에서 바로 편집 — 저장 즉시 defects.json에 반영(단, 이 편집 자체는 git 자동 커밋 대상 아님 — 잦은 소소한 변경이라 배치로 커밋)

🔗 Git 상태 · push

origin 대비 몇 커밋 앞서 있는지·최근 커밋 목록을 보여주고, 버튼을 눌러야만 push — 저장소 전체 기준(프로젝트 무관)

📄 Full TC 뷰어 — 체크박스 실행

TC 뷰어(HTML) 목록에서 TC를 체크하고 "선택 TC 테스트 실행" — 대시보드와 같은 세션 쿠키를 공유해 같은 테스트 실행 큐 엔진을 그 자리에서 호출

+ 새 프로젝트 / 관리 / 삭제

_template/ 복사로 신규 생성, project.json 필드 수정, 삭제 시 backup/deleted-projects/에 자동 백업 후 로컬에서만 제거(git 반영은 별도)

04 사용자 워크플로우

Slack 명령어는 더 이상 쓰지 않습니다. 아래는 실제로 쓰는 두 경로의 조작 방법입니다.

경로 A — 터미널/IDE 직접 세션

데모사이트 프로젝트 장바구니 TC 만들어

자연어 요청 — Phase 0(신규 프로젝트면)부터 순서대로 진행, 매 Phase 승인 필요

테스트 실행해줘

Phase 4 승인 후에만 유효 — 자동으로 실행되지 않고 반드시 별도 요청해야 Phase 5 시작

경로 B — 대시보드 조작

화면 요소	동작
채팅 패널에 메시지 입력	경로 A와 동일한 자연어 요청 (프로젝트 선택 상태에서)
테스트 실행 큐 → 필터 선택 → "▶ 테스트 실행 시작"	선택 조건에 맞는 TC를 순차 실행(zero-token)
"+ TC 업로드" → 파일 선택 → "업로드 및 반영"	엑셀/JSON을 캐노니컬 TC json에 즉시 반영 + 뷰어 재생성 자동 요청
결함 관리 표의 상태·담당자·이슈링크 셀 클릭	바로 편집, 저장 시 즉시 defects.json 반영
"  Git" → "origin으로 push"	쌓인 커밋을 원격에 반영 (매번 직접 클릭해야 함)